

LAPORAN PRATIKUM PEKAN 7
STRUKTUR DATA

Disusun Oleh:

Rifqi Aditya

2511533002

Dosen Pengampu :

Dr. Wahyudi, S.T, M.T

Asisten Praktikum :

Sherly Sukmadira



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
2026

KATA PENGANTAR

Puji syukur ke hadirat Allah SWT karena atas rahmat dan karunia-Nya saya dapat menyelesaikan laporan praktikum mata Struktur Data tentang *Sorting*.

Laporan ini disusun untuk memenuhi salah satu tugas praktikum sekaligus sebagai sarana pembelajaran dalam memahami dasar-dasar pemrograman menggunakan bahasa Java. Melalui praktikum ini, penulis dapat mempelajari penggunaan *Sorting*.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis mengucapkan terima kasih kepada dosen pengampu, asisten praktikum, serta rekan-rekan yang telah membantu dalam proses penyusunan laporan ini.

Akhir kata, semoga laporan ini dapat memberikan manfaat, baik bagi penulis sendiri maupun bagi pembaca, khususnya dalam memperdalam pemahaman tentang pemrograman dasar dengan Java.

Padang, 21 Mei 2025

Penulis

Daftar Pustaka

BAB I PENDAHULUAN.....	5
1.1 Latar Belakang	5
1.2 Tujuan	5
1.3 Manfaat.....	5
BAB II PEMBAHASAN.....	6
2.1 Insertion Sort.....	6
2.1.1 Kode Program	6
2.1.2 Uraian Kode Program.....	6
2.1.3 Output	7
2.2 Selection Sort.....	8
2.2.1 Kode Program	8
2.2.2 Uraian Kode Program.....	8
2.2.3 Output	9
2.3 Bubble Sort	9
2.3.1 Kode Program	9
2.3.2 Uraian Kode Program.....	9
2.3.3 Output	10
2.4 Insertion Sort GUI.....	10
2.4.1 Kode Program	10
2.4.2 Uraian Kode Program.....	12
2.4.3 Output	13
BAB III TUGAS	14
3.1 Kelas ADT.....	14
3.1.1 Kode Program	14
3.1.2 Uraian Kode Program.....	14
3.2 Method Sorting.....	15
3.2.1 Kode Program	15
3.2.2 Uraian Kode Program.....	16
3.3 GUI.....	18
3.3.1 Kode Program	18
3.3.2 Uraian Kode Program.....	19
3.4 Output.....	21

BAB III KESIMPULAN	24
---------------------------------	-----------

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pemrosesan data dan ilmu komputer, pengurutan data (*sorting*) merupakan operasi fundamental yang sangat penting untuk menyajikan informasi secara terstruktur. Data yang masih acak sering kali membatasi efisiensi program, terutama ketika sistem harus melakukan pencarian atau analisis informasi dalam jumlah besar. Oleh karena itu, penerapan algoritma *sorting* menjadi solusi utama untuk mengorganisasi elemen-elemen data, baik secara naik maupun menurun.

Terdapat berbagai macam pendekatan dalam melakukan pengurutan data, di mana *Bubble Sort*, *Selection Sort*, dan *Insertion Sort* merupakan tiga algoritma dasar yang paling sering dipelajari. *Bubble Sort* bekerja dengan cara menukar elemen yang bersebelahan secara berulang jika urutannya salah. *Selection Sort* beroperasi dengan mencari elemen nilai terkecil dari sekumpulan data dan menempatkannya di posisi awal. Sementara itu, *Insertion Sort* mengurutkan data dengan cara mengambil satu per satu elemen dan menyisipkannya ke posisi yang tepat, mirip seperti seseorang yang sedang mengurutkan kartu di tangannya.

1.2 Tujuan

1. Memahami konsep dasar dan mekanisme kerja dari algoritma pengurutan *Bubble Sort*, *Selection Sort*, dan *Insertion Sort*.
2. Mampu mengimplementasikan ketiga algoritma *sorting* tersebut ke dalam bahasa pemrograman untuk mengurutkan sekumpulan data acak.
3. Melatih kemampuan logika pemrograman dalam membandingkan, menukar posisi data (*swapping*), serta mengelola perulangan di dalam struktur *array*.

1.3 Manfaat

1. Meningkatkan pemahaman tentang cara kerja algoritma fundamental dan efisiensinya dalam manipulasi data linier.
2. Membantu mahasiswa dalam mengembangkan kemampuan pemrograman yang lebih terstruktur saat menangani penyusunan elemen data.
3. Meningkatkan kemampuan analisis dan pemecahan masalah (*problem-solving*) melalui penelusuran (*tracing*) proses pemindahan data pada masing-masing metode pengurutan.

BAB II

PEMBAHASAN

2.1 Insertion Sort

2.1.1 Kode Program

```
1 package Pekan7_2511533002;
2
3 public class InsertionSort_2511533002 {
4
5     public static void insertionSort_3002(int[] arr_3002) {
6         int n_3002 = arr_3002.length; // 'length' adalah bawaan Java, jadi tidak diubah
7         for (int i_3002 = 1; i_3002 < n_3002; i_3002++) {
8             int key_3002 = arr_3002[i_3002];
9             int j_3002 = i_3002 - 1;
10
11             while (j_3002 >= 0 && arr_3002[j_3002] > key_3002) {
12                 arr_3002[j_3002 + 1] = arr_3002[j_3002];
13                 j_3002--;
14             }
15             arr_3002[j_3002 + 1] = key_3002;
16         }
17     }
18
19     public static void main(String[] args_3002) {
20         int arr_3002[] = { 23, 78, 45, 8, 32, 56, 1 };
21         int n_3002 = arr_3002.length;
22
23         System.out.printf("array yang belum terurut:\n");
24         for (int i_3002 = 0; i_3002 < n_3002; i_3002++) {
25             System.out.print(arr_3002[i_3002] + " ");
26         }
27         System.out.println("");
28
29         insertionSort_3002(arr_3002);
30
31         System.out.printf("array yang terurut:\n");
32         for (int i_3002 = 0; i_3002 < n_3002; i_3002++) {
33             System.out.print(arr_3002[i_3002] + " ");
34         }
35         System.out.println("");
36     }
37 }
```

2.1.2 Uraian Kode Program

1. `for (int i_3002 = 1; i_3002 < n_3002; i_3002++) {` = Melakukan perulangan untuk menyisir elemen array dimulai dari indeks ke-1 hingga elemen terakhir.
2. `int key_3002 = arr_3002[i_3002];` = Mengambil nilai elemen pada indeks saat ini dan menyimpannya di variabel sementara `key_3002` sebagai angka yang akan dicari posisi tepatnya.
3. `int j_3002 = i_3002 - 1;` = Menentukan indeks variabel `j_3002` yang berada tepat satu posisi di sebelah kiri variabel `key_3002`.

4. **while (j_3002 >= 0 && arr_3002[j_3002] > key_3002)** { = Memulai perulangan mundur selama indeks j_3002 bernilai valid (0 atau lebih) DAN nilai elemen tersebut lebih besar dari nilai key_3002.
5. **arr_3002[j_3002 + 1] = arr_3002[j_3002];** = Menggeser nilai elemen yang lebih besar tersebut satu posisi ke sebelah kanan.
6. **j_3002--;** = Mengurangi nilai indeks j_3002 agar pada iterasi while berikutnya program mengecek elemen di sebelah kirinya lagi.
7. **arr_3002[j_3002 + 1] = key_3002;** = Menyisipkan nilai variabel key_3002 ke posisi yang benar setelah proses pergeseran angka selesai
8. **int arr_3002[] = { 23, 78, 45, 8, 32, 56, 1 };** = Membuat objek data array baru bernama arr_3002 yang berisi kumpulan angka acak.
9. **int n_3002 = arr_3002.length;** = Menghitung jumlah elemen yang ada di dalam array utama untuk proses cetak data.
10. **for (int i_3002 = 0; i_3002 < n_3002; i_3002++)** { = Melakukan perulangan untuk mengakses isi array dari awal (indeks 0) sampai akhir.
11. **for (int i_3002 = 0; i_3002 < n_3002; i_3002++)** { = Melakukan perulangan kembali untuk mengakses isi array yang posisinya sudah diperbarui.

2.1.3 Output

```
array yang belum terurut:  
23 78 45 8 32 56 1  
array yang terurut:  
1 8 23 32 45 56 78
```

2.2 Selection Sort

2.2.1 Kode Program

```
1 package Pekan7_2511533002;
2
3 public class SelectionSort_2511533002 {
4
5     public static void selectionSort_3002(int[] arr_3002) {
6         int n_3002 = arr_3002.length;
7         for (int i_3002 = 0; i_3002 < n_3002; i_3002++) {
8             int minIndex_3002 = i_3002;
9             for (int j_3002 = i_3002 + 1; j_3002 < n_3002; j_3002++) {
10                 if (arr_3002[j_3002] < arr_3002[minIndex_3002]) {
11                     minIndex_3002 = j_3002;
12                 }
13             }
14             int temp_3002 = arr_3002[i_3002];
15             arr_3002[i_3002] = arr_3002[minIndex_3002];
16             arr_3002[minIndex_3002] = temp_3002;
17         }
18     }
19
20     public static void main(String[] args_3002) {
21         int arr_3002[] = { 23, 78, 45, 8, 32, 56, 1 };
22         int n_3002 = arr_3002.length;
23
24         System.out.printf("array yang belum terurut:\n");
25         for (int i_3002 = 0; i_3002 < n_3002; i_3002++) {
26             System.out.print(arr_3002[i_3002] + " ");
27         }
28         System.out.println("");
29
30         selectionSort_3002(arr_3002);
31
32         System.out.printf("array yang terurut:\n");
33         for (int i_3002 = 0; i_3002 < n_3002; i_3002++) {
34             System.out.print(arr_3002[i_3002] + " ");
35         }
36         System.out.println("");
37     }
38 }
```

2.2.2 Uraian Kode Program

1. **for (int i_3002 = 0; i_3002 < n_3002; i_3002++)** { = Melakukan perulangan utama dari awal hingga akhir array untuk menentukan batas wilayah di mana kita akan menaruh angka terkecil.
2. **int minIndex_3002 = i_3002;** = Mengasumsikan indeks saat ini sebagai posisi yang menyimpan angka paling kecil sementara.
3. **for (int j_3002 = i_3002 + 1; j_3002 < n_3002; j_3002++)** { = Melakukan perulangan kedua untuk mengecek sisa elemen yang berada di sebelah kanan indeks i_3002.
4. **if (arr_3002[j_3002] < arr_3002[minIndex_3002])** { = Mengecek apakah elemen yang sedang diperiksa nilainya lebih kecil dibandingkan angka minimum sementara.
5. **minIndex_3002 = j_3002;** = Jika ditemukan angka yang lebih kecil, perbarui nilai minIndex_3002 dengan indeks dari angka yang baru tersebut.
6. **int temp_3002 = arr_3002[i_3002];** = Menyalin nilai elemen awal di posisi i_3002 ke dalam variabel sementara bernama temp_3002 agar tidak hilang saat ditimpa.
7. **arr_3002[i_3002] = arr_3002[minIndex_3002];** = Memindahkan angka paling kecil yang ditemukan ke posisi depan/awal sesuai iterasi saat ini.
8. **arr_3002[minIndex_3002] = temp_3002;** = Menaruh angka dari posisi awal tadi ke tempat asal angka terkecil.
9. **int arr_3002[] = { 23, 78, 45, 8, 32, 56, 1 };** = Membuat objek data array baru bernama arr_3002 yang berisi kumpulan angka acak.

10. `int n_3002 = arr_3002.length;` = Menghitung jumlah elemen yang ada di dalam array utama untuk proses cetak data.

2.2.3 Output

```
array yang belum terurut:
23 78 45 8 32 56 1
array yang terurut:
1 8 23 32 45 56 78
```

2.3 Bubble Sort

2.3.1 Kode Program

```
1 package Pekan7_2511533002;
2
3 public class BubbleSort_2511533002 {
4
5     public static void bubbleSort_3002(int[] arr_3002) {
6         int n_3002 = arr_3002.length;
7         for (int i_3002 = 0; i_3002 < n_3002; i_3002++) {
8             for (int j_3002 = 0; j_3002 < n_3002 - i_3002 - 1; j_3002++) {
9                 if (arr_3002[j_3002] > arr_3002[j_3002 + 1]) {
10                     int temp_3002 = arr_3002[j_3002];
11                     arr_3002[j_3002] = arr_3002[j_3002 + 1];
12                     arr_3002[j_3002 + 1] = temp_3002;
13                     // System.out.println("data:" + arr_3002[j_3002] + " " + arr_3002[j_3002 + 1]);
14                 }
15             }
16         }
17     }
18
19     public static void main(String[] args_3002) {
20         int arr_3002[] = { 23, 78, 45, 8, 32, 56, 1 };
21         int n_3002 = arr_3002.length;
22
23         System.out.print("array yang belum terurut:");
24         for (int i_3002 = 0; i_3002 < n_3002; i_3002++) {
25             System.out.print(arr_3002[i_3002] + " ");
26         }
27         System.out.println("");
28
29         bubbleSort_3002(arr_3002);
30
31         System.out.print("array yang terurut menggunakan BubleSort:");
32         for (int i_3002 = 0; i_3002 < n_3002; i_3002++) {
33             System.out.print(arr_3002[i_3002] + " ");
34         }
35         System.out.println("");
36     }
37 }
```

2.3.2 Uraian Kode Program

1. `for (int i_3002 = 0; i_3002 < n_3002; i_3002++) {` = Melakukan perulangan utama untuk mengontrol berapa kali keseluruhan proses penyisiran data harus diulang.
2. `for (int j_3002 = 0; j_3002 < n_3002 - i_3002 - 1; j_3002++) {` = Melakukan perulangan di dalam untuk mengecek pasangan elemen yang bersebelahan. Batas perulangan dikurangi `i_3002` karena setiap putaran selesai, angka terbesar akan mengapung ke posisi paling belakang, sehingga tidak perlu dicek ulang.
3. `if (arr_3002[j_3002] > arr_3002[j_3002 + 1]) {` = Mengecek kondisi apakah nilai elemen di posisi sebelah kiri lebih besar daripada nilai elemen tepat di sebelah kanannya. Jika ya, posisinya salah dan harus ditukar.

4. **int temp_3002 = arr_3002[j_3002];** = Menyimpan nilai dari elemen sebelah kiri ke dalam variabel sementara bernama temp_3002 agar nilainya tidak hilang saat ditimpa.
5. **arr_3002[j_3002] = arr_3002[j_3002 + 1];** = Memasukkan nilai dari elemen sebelah kanan ke posisi sebelah kiri.
6. **arr_3002[j_3002 + 1] = temp_3002;** = Memasukkan nilai elemen yang sebelumnya di kiri ke posisi sebelah kanan.
7. **int arr_3002[] = { 23, 78, 45, 8, 32, 56, 1 };** = Membuat objek data array baru bernama arr_3002 yang berisi kumpulan angka acak.
8. **int n_3002 = arr_3002.length;** = Menghitung jumlah elemen yang ada di dalam array utama untuk proses perulangan cetak data.
9. **for (int i_3002 = 0; i_3002 < n_3002; i_3002++)** { = Melakukan perulangan untuk mengakses isi array dari awal sampai akhir.
10. **bubbleSort_3002(arr_3002);** = Memanggil fungsi pengurutan dan mengirimkan data arr_3002 agar diproses menggunakan metode Bubble Sort.
11. **for (int i_3002 = 0; i_3002 < n_3002; i_3002++)** { = Melakukan perulangan kembali untuk mengakses isi array yang kini susunannya sudah rapi dari terkecil hingga terbesar.

2.3.3 Output

```
array yang belum terurut:23 78 45 8 32 56 1
array yang terurut menggunakan BubleSort:1 8 23 32 45 56 78
```

2.4 Insertion Sort GUI

2.4.1 Kode Program

```
21 public class InsertionSortGUI_2511533002 extends JFrame {
22     private static final long serialVersionUID = 1L;
23     private int[] array_3002;
24     private JLabel[] labelArray_3002;
25     private JButton stepButton_3002, resetButton_3002, setButton_3002;
26     private JTextField inputField_3002;
27     private JPanel panelArray_3002;
28     private JTextArea stepArea_3002;
29
30     private int i = 1, j;
31     private boolean sorting_3002 = false;
32     private int stepCount_3002 = 1;
33
34     /**
35      * Create the frame.
36      */
37     public InsertionSortGUI_2511533002() {
38         setTitle("Insertion Sort Langkah per Langkah");
39         setSize(750, 400);
40         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
41         setLocationRelativeTo(null);
42         setLayout(new BorderLayout());
43
44         //Panel Input
45         JPanel inputPanel = new JPanel(new FlowLayout());
46         inputField_3002 = new JTextField(30);
47         setButton_3002 = new JButton("Set Array");
48         inputPanel.add(new JLabel("Masukkan angka (pisahkan dengan koma :)"));
49         inputPanel.add(inputField_3002);
50         inputPanel.add(setButton_3002);
51
52         //Panel Array Visual
53         panelArray_3002 = new JPanel();
54         panelArray_3002.setLayout(new FlowLayout());
```

```

56 //Panel Kontrol
57 JPanel controlPanel_3002 = new JPanel();
58 stepButton_3002 = new JButton("Langkah Selanjutnya");
59 resetButton_3002 = new JButton("Reset");
60 stepButton_3002.setEnabled(false);
61 controlPanel_3002.add(stepButton_3002);
62 controlPanel_3002.add(resetButton_3002);
63
64 //Area teks untuk log langkah langkah
65 JTextArea stepArea_3002 = new JTextArea(8, 60);
66 stepArea_3002.setEditable(false);
67 stepArea_3002.setFont(new Font("Monospaced", Font.PLAIN, 14));
68 JScrollPane scrollPane_3002 = new JScrollPane(stepArea_3002);
69
70 // Tambahkan panel ke frame
71 add(inputPanel, BorderLayout.NORTH);
72 add(panelArray_3002, BorderLayout.CENTER);
73 add(controlPanel_3002, BorderLayout.SOUTH);
74 add(scrollPane_3002, BorderLayout.EAST);
75
76 //Event Set Array
77 setButton_3002.addActionListener(e -> setArrayFromInput_3002());
78
79 //Event Langkah Selanjutnya
80 stepButton_3002.addActionListener(e -> performStep_3002());
81
82 // Event Reset
83 resetButton_3002.addActionListener(e -> reset());
84 }
85
86 private void setArrayFromInput_3002() {
87     String text_3002 = inputField_3002.getText().trim();
88     if (text_3002.isEmpty()) return;
89     String[] parts_3002 = text_3002.split(",");
90     array_3002 = new int[parts_3002.length];

```

```

91     try {
92         for (int k = 0; k < parts_3002.length; k++) {
93             array_3002[k] = Integer.parseInt(parts_3002[k].trim());
94         }
95     } catch (NumberFormatException e) {
96         JOptionPane.showMessageDialog(this, "Masukkan hanya angka yang dipisahkan dengan koma", "Error", JOptionPane.ERROR_MESSAGE);
97         return;
98     }
99
100 // --- PERBAIKAN DI SINI ---
101 sorting_3002 = true; // Mengubah status agar bisa di-sorting
102 // -----
103
104 i = 1;
105 stepCount_3002 = 1;
106 stepButton_3002.setEnabled(true);
107 stepArea_3002.setText("");
108 panelArray_3002.removeAll();
109 labelArray_3002 = new JLabel[array_3002.length];
110 for (int k = 0; k < array_3002.length; k++) {
111     labelArray_3002[k] = new JLabel(String.valueOf(array_3002[k]));
112     labelArray_3002[k].setFont(new Font("Arial", Font.BOLD, 24));
113     labelArray_3002[k].setBorder(BorderFactory.createLineBorder(Color.BLACK));
114     labelArray_3002[k].setPreferredSize(new Dimension(50, 50));
115     labelArray_3002[k].setHorizontalAlignment(SwingConstants.CENTER);
116     panelArray_3002.add(labelArray_3002[k]);
117 }
118 panelArray_3002.revalidate();
119 panelArray_3002.repaint();
120 }
121
122 private void performStep_3002() {
123     if (i < array_3002.length && sorting_3002) {
124         int key_3002 = array_3002[i];
125         j = i - 1;

```

```

127     StringBuilder stepLog_3002 = new StringBuilder();
128     stepLog_3002.append("Langkah ").append(stepCount_3002).append(" Memasukkan ").append(key_3002).append("\n\n");
129
130     while (j >= 0 && array_3002[j] > key_3002) {
131         array_3002[j + 1] = array_3002[j];
132         j--;
133     }
134     array_3002[j + 1] = key_3002;
135
136     updateLabels();
137     stepLog_3002.append("Hasil: ").append(arrayToString(array_3002)).append("\n\n");
138     stepArea_3002.append(stepLog_3002.toString());
139
140     i++;
141     stepCount_3002++;
142
143     if (i == array_3002.length) {
144         sorting_3002 = false;
145         stepButton_3002.setEnabled(false);
146         JOptionPane.showMessageDialog(this, "Sorting selesai!");
147     }
148 }
149
150 private void updateLabels() {
151     for (int k = 0; k < array_3002.length; k++) {
152         labelArray_3002[k].setText(String.valueOf(array_3002[k]));
153     }
154 }
155
156 private void reset() {
157     inputField_3002.setText("");
158     panelArray_3002.removeAll();
159     panelArray_3002.revalidate();
160     panelArray_3002.repaint();
161 }

```

```

162     stepArea_3002.setText("");
163     stepButton_3002.setEnabled(false);
164     sorting_3002 = false;
165     i = 1;
166     stepCount_3002 = 1;
167 }
168
169 private String arrayToString(int[] arr_3002) {
170     StringBuilder sb = new StringBuilder();
171     for (int k = 0; k < arr_3002.length; k++) {
172         sb.append(arr_3002[k]);
173         if (k < arr_3002.length - 1) sb.append(", ");
174     }
175     return sb.toString();
176 }
177
178 public static void main(String[] args) {
179     SwingUtilities.invokeLater(() -> {
180         InsertionSortGUI_2511533002 gui = new InsertionSortGUI_2511533002();
181         gui.setVisible(true);
182     });
183 }
184 }

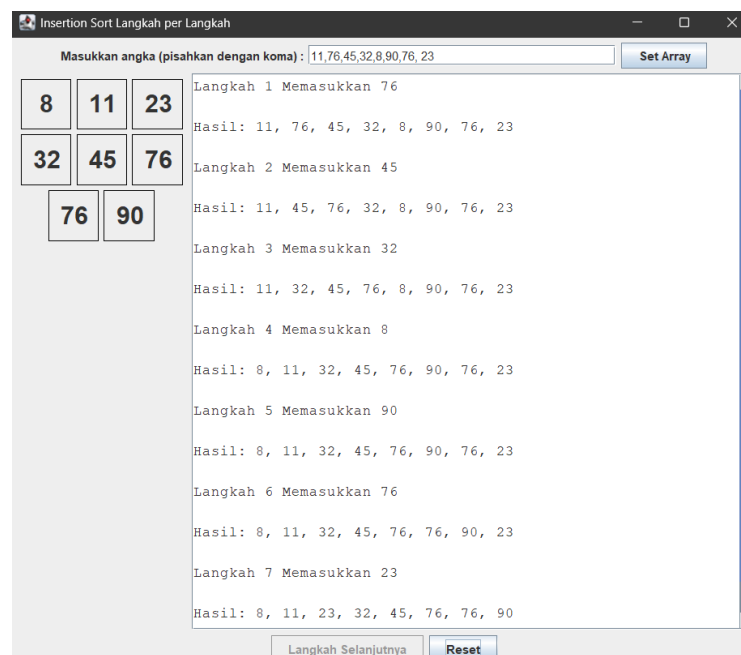
```

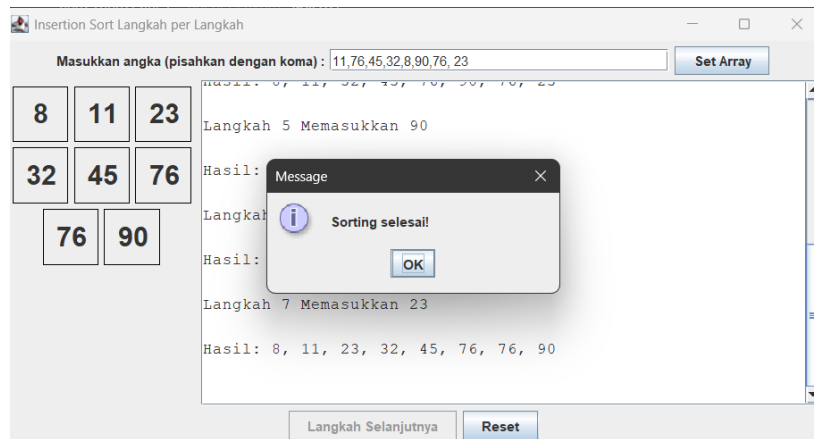
2.4.2 Uraian Kode Program

- setTitle("Insertion Sort Langkah per Langkah");**
setSize(750, 400);
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
setLocationRelativeTo(null); = Mengatur judul jendela, ukuran (750x400 piksel), aksi saat tombol x ditekan, dan membuat jendela muncul tepat di tengah layar komputer.
- setLayout(new BorderLayout());** = Mengatur tata letak utama jendela menjadi BorderLayout.
- JPanel inputPanel = new JPanel(new FlowLayout());** = Membuat panel penampung untuk area input teks dan menatanya secara berjejer.
- stepButton_3002.setEnabled(false);** = Menonaktifkan tombol "Langkah Selanjutnya" di awal sebelum ada data yang dimasukkan.
- stepArea_3002 = new JTextArea(8, 60);** = Membuat area teks berukuran 8 baris dan lebar 60 karakter untuk mencatat riwayat langkah pengurutan.
- JScrollPane scrollPane_3002 = new JScrollPane(stepArea_3002);** = Menambahkan fitur scroll pada area teks log jika teksnya sudah terlalu panjang.
- setButton_3002.addActionListener(e -> setArrayFromInput_3002());** = Memberikan perintah agar saat tombol "Set Array" diklik, program menjalankan method `setArrayFromInput_3002()`.
- String text_3002 = inputField_3002.getText().trim();** = Mengambil teks yang diketik pengguna dan menghapus spasi berlebih di awal/akhir teks.
- String[] parts_3002 = text_3002.split(",");** = Memecah teks tersebut menjadi bagian-bagian kecil (berupa array String) berdasarkan tanda koma.
- array_3002 = new int[parts_3002.length];** = Menyiapkan array integer kosong dengan panjang sesuai jumlah angka yang dimasukkan.
- try{for(int k=0;k<parts_3002.length;k++)**
 {array_3002[k]=Integer.parseInt(parts_3002[k].trim());
 }} catch (NumberFormatException e)
 { JOptionPane.showMessageDialog(this,"Masukkan hanya angka yang dipisahkan dengan koma!","Error",JOptionPane.ERROR_MESSAGE);
 return;} = Mencoba mengubah teks menjadi angka. Jika pengguna memasukkan huruf, program akan menangkap pesan dan memunculkan pop-up error.
- sorting_3002 = true;** = Mengaktifkan status bahwa program siap melakukan pengurutan.
- panelArray_3002.removeAll();** = Membersihkan panel visualisasi dari kotak angka sebelumnya.
- labelArray_3002[k] = new JLabel(String.valueOf(array_3002[k]));** = Membuat elemen kotak label untuk setiap angka yang valid.

15. **panelArray_3002.add(labelArray_3002[k]);** = Menambahkan kotak-kotak tersebut ke dalam layar.
16. **panelArray_3002.revalidate(); panelArray_3002.repaint();** = Menyegarkan (refresh) tampilan antarmuka agar kotak-kotak baru tersebut langsung muncul.
17. **if(i < array_3002.length && sorting_3002)** { = Memastikan langkah ini hanya berjalan jika proses belum selesai dan mode pengurutan aktif.
18. **int key_3002 = array_3002[i]; j = i - 1;** = Mengambil nilai key (angka yang akan dipindahkan) dari indeks ke-i dan menetapkan penunjuk j ke sebelah kirinya.
19. **StringBuilder stepLog_3002 = new StringBuilder();** = Menyiapkan objek pembuat teks untuk menulis catatan (log) langkah saat ini ke area teks kanan.
20. **updateLabels();** = Memanggil method khusus untuk memperbarui angka di dalam kotak GUI sesuai posisi array terbaru.
21. **stepArea_3002.append(stepLog_3002.toString());** = Memasukkan teks hasil rekaman langkah tadi ke dalam layar log.
22. **i++; stepCount_3002++;** = Menaikkan nilai indeks dan nomorurut langkah untuk persiapan klik tombol "Langkah Selanjutnya" berikutnya.
23. **SwingUtilities.invokeLater() -> { ... };** = Perintah standar dan aman di Java untuk memastikan bahwa program antarmuka grafis (GUI) dijalankan pada thread (jalur eksekusi) khusus antarmuka.
24. **gui.setVisible(true);** = Membuat jendela aplikasi InsertionSortGUI_2511533002 muncul dan bisa dilihat oleh pengguna di layar monitor.

2.4.3 Output





BAB III TUGAS

3.1 Kelas ADT

3.1.1 Kode Program

```

1 package Pekan7_2511533002;
2
3 public class Mahasiswa_2511533002 {
4     private String nama_3002;
5     private String nim_3002;
6     private String prodi_3002;
7
8     public Mahasiswa_2511533002(String nama_3002, String nim_3002, String prodi_3002) {
9         this.nama_3002 = nama_3002;
10        this.nim_3002 = nim_3002;
11        this.prodi_3002 = prodi_3002;
12    }
13
14    public String getNama_3002() { return nama_3002; }
15    public void setNama_3002(String nama_3002) { this.nama_3002 = nama_3002; }
16
17    public String getNim_3002() { return nim_3002; }
18    public void setNim_3002(String nim_3002) { this.nim_3002 = nim_3002; }
19
20    public String getProdi_3002() { return prodi_3002; }
21    public void setProdi_3002(String prodi_3002) { this.prodi_3002 = prodi_3002; }
22
23    @Override
24    public String toString() {
25        return nama_3002 + " | " + nim_3002 + " | " + prodi_3002;
26    }
27 }

```

3.1.2 Uraian Kode Program

1. **private String nama_3002;** = Mendeklarasikan variabel nama_3002 bertipe teks String dengan hak akses tertutup , artinya hanya bisa dimodifikasi dari dalam kelas ini saja.
2. **private String nim_3002;** = Mendeklarasikan variabel nim_3002 bertipe teks String dengan hak akses tertutup private untuk menyimpan data NIM.

3. **private String prodi_3002;** = Mendeklarasikan variabel prodi_3002 bertipe teks String dengan hak akses tertutup private untuk menyimpan data Program Studi.
4. **public Mahasiswa_2511533002(String nama_3002, String nim_3002, String prodi_3002) {** = Membuat metode konstruktor yang akan otomatis dijalankan saat objek Mahasiswa baru diciptakan. Metode ini mewajibkan pengisian tiga nilai awal (nama, nim, dan prodi).
5. **this.nama_3002 = nama_3002;** = Menugaskan nilai dari parameter nama_3002 untuk disimpan ke dalam variabel atribut milik objek itu sendiri.
6. **this.nim_3002 = nim_3002;** = Menugaskan nilai dari parameter nim_3002 ke dalam variabel atribut nim_3002 milik objek.
7. **this.prodi_3002 = prodi_3002;** = Menugaskan nilai dari parameter prodi_3002 ke dalam variabel atribut prodi_3002 milik objek. **public String toString() {** = Mendeklarasikan metode toString yang menentukan format bagaimana objek mahasiswa ini ditampilkan jika dicetak.
8. **return nama_3002 + " | " + nim_3002 + " | " + prodi_3002;** = Menggabungkan teks dari atribut nama, nim, dan prodi yang masing-masing dipisahkan oleh simbol garis (|), kemudian mengembalikan hasil gabungan tersebut.

3.2 Method Sorting

3.2.1 Kode Program

```

1 package Pekan7_2511533002;
2
3 import java.util.ArrayList;
4 import javax.swing.JTextArea;
5
6 public class MahasiswaSort_2511533002 {
7
8     public static String getListNama_3002(ArrayList<Mahasiswa_2511533002> list_3002) {
9         StringBuilder sb_3002 = new StringBuilder("");
10        for (int i_3002 = 0; i_3002 < list_3002.size(); i_3002++) {
11            sb_3002.append(list_3002.get(i_3002).getNama_3002());
12            if (i_3002 < list_3002.size() - 1) sb_3002.append(", ");
13        }
14        sb_3002.append("\n");
15        return sb_3002.toString();
16    }
17
18    public static void insertionSort_3002(ArrayList<Mahasiswa_2511533002> arr_3002, JTextArea areaProses_3002) {
19        for (int i_3002 = 1; i_3002 < arr_3002.size(); i_3002++) {
20            Mahasiswa_2511533002 key_3002 = arr_3002.get(i_3002);
21            int j_3002 = i_3002 - 1;
22
23            while (j_3002 >= 0 && arr_3002.get(j_3002).getNama_3002().compareToIgnoreCase(key_3002.getNama_3002()) > 0) {
24                arr_3002.set(j_3002 + 1, arr_3002.get(j_3002));
25                j_3002--;
26            }
27            arr_3002.set(j_3002 + 1, key_3002);
28            areaProses_3002.append("Langkah " + i_3002 + ": " + getListNama_3002(arr_3002) + "\n");
29        }
30    }
31
32    public static void selectionSort_3002(ArrayList<Mahasiswa_2511533002> arr_3002, JTextArea areaProses_3002) {
33        for (int i_3002 = 0; i_3002 < arr_3002.size() - 1; i_3002++) {
34            int minIdx_3002 = i_3002;
35            for (int j_3002 = i_3002 + 1; j_3002 < arr_3002.size(); j_3002++) {
36                if (arr_3002.get(j_3002).getNama_3002().compareToIgnoreCase(arr_3002.get(minIdx_3002).getNama_3002()) < 0) {
37                    minIdx_3002 = j_3002;

```

```

38     }
39 }
40
41 Mahasiswa_2511533002 temp_3002 = arr_3002.get(minIdx_3002);
42 arr_3002.set(minIdx_3002, arr_3002.get(i_3002));
43 arr_3002.set(i_3002, temp_3002);
44
45 areaProses_3002.append("Pass " + (i_3002 + 1) + ": " + getListNama_3002(arr_3002) + "\n");
46 }
47 }
48
49 public static void bubbleSort_3002(ArrayList<Mahasiswa_2511533002> arr_3002, JTextArea areaProses_3002) {
50     for (int i_3002 = 0; i_3002 < arr_3002.size() - 1; i_3002++) {
51         boolean swapped_3002 = false;
52         for (int j_3002 = 0; j_3002 < arr_3002.size() - i_3002 - 1; j_3002++) {
53             if (arr_3002.get(j_3002).getNama_3002().compareToIgnoreCase(arr_3002.get(j_3002 + 1).getNama_3002()) > 0) {
54                 // Swap
55                 Mahasiswa_2511533002 temp_3002 = arr_3002.get(j_3002);
56                 arr_3002.set(j_3002, arr_3002.get(j_3002 + 1));
57                 arr_3002.set(j_3002 + 1, temp_3002);
58                 swapped_3002 = true;
59             }
60         }
61         areaProses_3002.append("Pass " + (i_3002 + 1) + ": " + getListNama_3002(arr_3002) + "\n");
62         if (!swapped_3002) break;
63     }
64 }
65 }

```

3.2.2 Uraian Kode Program

1. **StringBuilder sb_3002 = new StringBuilder("");** = Membuat objek StringBuilder untuk merangkai teks secara efisien, diawali dengan kurung siku buka.
2. **for (int i_3002 = 0; i_3002 < list_3002.size(); i_3002++)** { = Melakukan perulangan untuk menelusuri setiap elemen di dalam ArrayList dari awal hingga akhir.
3. **sb_3002.append(list_3002.get(i_3002).getNama_3002());** = Mengambil nama mahasiswa pada indeks saat ini dan menambahkannya ke dalam rangkaian teks.
4. **if (i_3002 < list_3002.size() - 1) sb_3002.append(", ");** = Menambahkan tanda koma dan spasi jika elemen tersebut bukanlah elemen yang terakhir.
5. **sb_3002.append("");** = Menambahkan kurung siku tutup pada akhir rangkaian teks.
6. **public static void insertionSort_3002(ArrayList<Mahasiswa_2511533002> arr_3002, JTextArea areaProses_3002)** { = Mendeklarasikan metode untuk melakukan pengurutan dengan algoritma Insertion Sort.
7. **for (int i_3002 = 1; i_3002 < arr_3002.size(); i_3002++)** { = Perulangan dimulai dari indeks ke-1, mengasumsikan elemen ke-0 sudah pada tempatnya.
8. **Mahasiswa_2511533002 key_3002 = arr_3002.get(i_3002);** = Menyimpan objek mahasiswa saat ini ke dalam variabel sementara key_3002.
9. **int j_3002 = i_3002 - 1;** = Menginisialisasi indeks j_3002 untuk menunjuk ke elemen tepat di sebelah kiri dari elemen yang sedang dievaluasi.
10. **while (j_3002 >= 0 && arr_3002.get(j_3002).getNama_3002().compareToIgnoreCase(key_3002.getNama_3002()) > 0)** { = Melakukan perulangan mundur selama elemen di sebelah kiri lebih besar secara alfabet.
11. **arr_3002.set(j_3002 + 1, arr_3002.get(j_3002));** = Menggeser elemen yang lebih besar tersebut satu posisi ke sebelah kanan.
12. **j_3002--;** = Mengurangi nilai indeks untuk mengecek elemen di sebelah kirinya lagi.
13. **arr_3002.set(j_3002 + 1, key_3002);** = Menempatkan objek key_3002 pada posisi yang benar setelah proses pergeseran selesai.

14. **areaProses_3002.append("Langkah " + i_3002 + ": " + getListNama_3002(arr_3002) + "\n");** = Menambahkan teks visualisasi langkah pengurutan saat ini ke komponen GUI.
15. **for (int i_3002 = 0; i_3002 < arr_3002.size() - 1; i_3002++)** { = Perulangan untuk menentukan posisi target di mana nilai terkecil akan ditempatkan.
16. **int minIdx_3002 = i_3002;** = Menetapkan indeks saat ini sebagai indeks dengan nilai nama terkecil sementara.
17. **for (int j_3002 = i_3002 + 1; j_3002 < arr_3002.size(); j_3002++)** { = Perulangan kedua untuk menelusuri sisa elemen di sebelah kanan guna mencari nilai yang lebih kecil.
18. **if**
(arr_3002.get(j_3002).getNama_3002().compareToIgnoreCase(arr_3002.get(minIdx_3002).getNama_3002()) < 0) { = Mengecek jika nama pada elemen saat ini lebih kecil secara alfabet dibandingkan nama pada posisi minimum sementara.
19. **minIdx_3002 = j_3002;** = Memperbarui indeks nilai terkecil jika ditemukan elemen dengan nama yang lebih awal secara alfabet.
20. **Mahasiswa_2511533002 temp_3002 = arr_3002.get(minIdx_3002);** = Menyimpan objek mahasiswa terkecil ke dalam variabel sementara untuk proses penukaran.
21. **arr_3002.set(minIdx_3002, arr_3002.get(i_3002));** = Memindahkan objek di posisi awal ke posisi di mana objek terkecil sebelumnya berada.
22. **arr_3002.set(i_3002, temp_3002);** = Menempatkan objek terkecil dari variabel sementara ke posisi target di urutan awal.
23. **areaProses_3002.append("Pass " + (i_3002 + 1) + ": " + getListNama_3002(arr_3002) + "\n");** = Menambahkan teks visualisasi hasil pertukaran ke komponen GUI.
24. **for (int i_3002 = 0; i_3002 < arr_3002.size() - 1; i_3002++)** { = Perulangan luar untuk menentukan jumlah putaran penyisiran array.
25. **boolean swapped_3002 = false;** = Mendeklarasikan variabel penanda untuk melacak apakah terjadi pertukaran elemen pada putaran ini.
26. **for (int j_3002 = 0; j_3002 < arr_3002.size() - i_3002 - 1; j_3002++)** { = Perulangan dalam untuk membandingkan dua elemen yang posisinya saling bersebelahan.

3.3 GUI

3.3.1 Kode Program

```
1 package Pekan7_2511533002;
2
3 import javax.swing.*;
4 import java.awt.*;
5 import java.util.ArrayList;
6
7 public class MahasiswaSortGui_2511533002 extends JFrame {
8     private static final long serialVersionUID = 1L;
9
10    private ArrayList<Mahasiswa_2511533002> dataMahasiswa_3002;
11
12    private JTextField txtNama_3002, txtNim_3002, txtProdi_3002;
13    private JButton btnTambah_3002, btnHapus_3002, btnSort_3002;
14    private JComboBox<String> comboAlgo_3002;
15    private JTextArea areaData_3002, areaProses_3002;
16
17    public MahasiswaSortGui_2511533002() {
18        dataMahasiswa_3002 = new ArrayList<>();
19
20        setTitle("Pengurutan Nama Mahasiswa");
21        setSize(850, 550);
22        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
23        setLocationRelativeTo(null);
24        setLayout(new BorderLayout(10, 10));
25
26        JPanel panelInput_3002 = new JPanel(new GridLayout(4, 2, 5, 5));
27        panelInput_3002.setBorder(BorderFactory.createTitledBorder("Input Data Mahasiswa"));
28
29        panelInput_3002.add(new JLabel("Nama:"));
30        txtNama_3002 = new JTextField();
31        panelInput_3002.add(txtNama_3002);
32
33        panelInput_3002.add(new JLabel("NIM:"));
34        txtNim_3002 = new JTextField();
35        panelInput_3002.add(txtNim_3002);
```

```
37        panelInput_3002.add(new JLabel("Program Studi:"));
38        txtProdi_3002 = new JTextField();
39        panelInput_3002.add(txtProdi_3002);
40
41        btnTambah_3002 = new JButton("Tambah Data");
42        btnHapus_3002 = new JButton("Hapus Data Terakhir");
43        panelInput_3002.add(btnTambah_3002);
44        panelInput_3002.add(btnHapus_3002);
45
46        JPanel panelKontrol_3002 = new JPanel(new FlowLayout(FlowLayout.CENTER, 15, 10));
47        panelKontrol_3002.setBorder(BorderFactory.createTitledBorder("Pilih Algoritma"));
48
49        panelKontrol_3002.add(new JLabel("Algoritma:"));
50        String[] algos_3002 = {"Insertion Sort", "Selection Sort", "Bubble Sort"};
51        comboAlgo_3002 = new JComboBox<>(algos_3002);
52        panelKontrol_3002.add(comboAlgo_3002);
53
54        btnSort_3002 = new JButton("Mulai Sorting");
55        panelKontrol_3002.add(btnSort_3002);
56
57        JPanel panelTampil_3002 = new JPanel(new GridLayout(1, 2, 10, 0));
58
59        areaData_3002 = new JTextArea();
60        areaData_3002.setEditable(false);
61        JScrollPane scrollData_3002 = new JScrollPane(areaData_3002);
62        scrollData_3002.setBorder(BorderFactory.createTitledBorder("Data Mahasiswa"));
63
64        areaProses_3002 = new JTextArea();
65        areaProses_3002.setEditable(false);
66        areaProses_3002.setFont(new Font("Monospaced", Font.PLAIN, 12));
67        JScrollPane scrollProses_3002 = new JScrollPane(areaProses_3002);
68        scrollProses_3002.setBorder(BorderFactory.createTitledBorder("Proses Sorting"));
```

```

70     panelTampil_3002.add(scrollData_3002);
71     panelTampil_3002.add(scrollProses_3002);
72
73     add(panelInput_3002, BorderLayout.WEST);
74     add(panelTampil_3002, BorderLayout.CENTER);
75     add(panelKontrol_3002, BorderLayout.SOUTH);
76
77     btnTambah_3002.addActionListener(e -> tambahData_3002());
78     btnHapus_3002.addActionListener(e -> hapusData_3002());
79     btnSort_3002.addActionListener(e -> eksekusiSorting_3002());
80 }
81
82 private void tambahData_3002() {
83     String nama_3002 = txtNama_3002.getText().trim();
84     String nim_3002 = txtNim_3002.getText().trim();
85     String prodi_3002 = txtProdi_3002.getText().trim();
86
87     if (nama_3002.isEmpty() || nim_3002.isEmpty() || prodi_3002.isEmpty()) {
88         JOptionPane.showMessageDialog(this, "Semua kolom harus diisi!", "Peringatan", JOptionPane.WARNING_MESSAGE);
89         return;
90     }
91
92     dataMahasiswa_3002.add(new Mahasiswa_2511533002(nama_3002, nim_3002, prodi_3002));
93     updateTampilanData_3002();
94
95     txtNama_3002.setText(""); txtNim_3002.setText(""); txtProdi_3002.setText("");
96     txtNama_3002.requestFocus();
97 }
98
99 private void hapusData_3002() {
100     if (!dataMahasiswa_3002.isEmpty()) {
101         dataMahasiswa_3002.remove(dataMahasiswa_3002.size() - 1);
102         updateTampilanData_3002();
103     } else {
104         JOptionPane.showMessageDialog(this, "Data masih kosong!");
105     }
106 }
107
108 private void updateTampilanData_3002() {
109     StringBuilder sb_3002 = new StringBuilder();
110     for (int i_3002 = 0; i_3002 < dataMahasiswa_3002.size(); i_3002++) {
111         sb_3002.append((i_3002 + 1)).append(". ").append(dataMahasiswa_3002.get(i_3002).toString()).append("\n");
112     }
113     areaData_3002.setText(sb_3002.toString());
114 }
115
116 private void eksekusiSorting_3002() {
117     if (dataMahasiswa_3002.isEmpty()) {
118         JOptionPane.showMessageDialog(this, "Tidak ada data untuk disorting!");
119         return;
120     }
121
122     ArrayList<Mahasiswa_2511533002> dataCopy_3002 = new ArrayList<>(dataMahasiswa_3002);
123     String pilihan_3002 = (String) comboAlgo_3002.getSelectedItem();
124
125     areaProses_3002.setText("=== " + pilihan_3002.toUpperCase() + " ===\n");
126     areaProses_3002.append("Awal: " + MahasiswaSort_2511533002.getListNama_3002(dataCopy_3002) + "\n\n");
127
128     if (pilihan_3002.equals("Insertion Sort")) {
129         MahasiswaSort_2511533002.insertionSort_3002(dataCopy_3002, areaProses_3002);
130     } else if (pilihan_3002.equals("Selection Sort")) {
131         MahasiswaSort_2511533002.selectionSort_3002(dataCopy_3002, areaProses_3002);
132     } else if (pilihan_3002.equals("Bubble Sort")) {
133         MahasiswaSort_2511533002.bubbleSort_3002(dataCopy_3002, areaProses_3002);
134     }
135
136     areaProses_3002.append("\n=== HASIL AKHIR ===\n");
137     for (Mahasiswa_2511533002 m_3002 : dataCopy_3002) {
138         areaProses_3002.append("- " + m_3002.toString() + "\n");
139     }
140 }
141
142 public static void main(String[] args_3002) {
143     SwingUtilities.invokeLater(() -> {

```

3.3.2 Uraian Kode Program

1. **setSize(850, 550);** = Mengatur ukuran resolusi jendela aplikasi menjadi lebar 850 piksel dan tinggi 550 piksel.
2. **setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);** = Menginstruksikan program untuk benar-benar berhenti beroperasi apabila tombol silang di sudut jendela ditekan.
3. **setLocationRelativeTo(null);** = Membuat jendela aplikasi muncul tepat di tengah layar monitor pengguna.

4. **setLayout(new BorderLayout(10, 10));** = Menerapkan tata letak BorderLayout dengan jarak antar komponen 10 piksel horizontal dan vertikal.
5. **JPanel panelInput_3002 = new JPanel(new GridLayout(4, 2, 5, 5));** = Membuat panel penampung form input dengan bentuk tabel grid berukuran 4 baris dan 2 kolom.
6. **panelInput_3002.setBorder(BorderFactory.createTitledBorder("Input Data Mahasiswa"));** = Menambahkan bingkai garis dan judul pada kotak form input tersebut.
7. **panelInput_3002.add(new JLabel("Nama:"));** = Menempatkan teks tulisan statis "Nama:" ke dalam kotak panel.
8. **txtNama_3002 = new JTextField();** = Membuat objek nyata dari kolom input teks untuk isian nama.
9. **btnTambah_3002 = new JButton("Tambah Data");** = Membuat tombol dengan label "Tambah Data".
10. **btnHapus_3002 = new JButton("Hapus Data Terakhir");** = Membuat tombol dengan label "Hapus Data Terakhir".
11. **panelInput_3002.add(btnTambah_3002);**
panelInput_3002.add(btnHapus_3002); = Memasukkan kedua tombol tersebut ke bagian bawah dalam panel grid input.
12. **JPanel panelKontrol_3002 = new JPanel(new**
FlowLayout(FlowLayout.CENTER, 15, 10)); = Membuat panel khusus kontrol dengan tata letak mengalir
13. **panelKontrol_3002.setBorder(BorderFactory.createTitledBorder("Pilih Algoritma"));** = Memberikan bingkai dengan teks judul "Pilih Algoritma" pada panel kontrol.
14. **comboAlgo_3002 = new JComboBox<>(algos_3002);** = Membuat elemen drop-down yang diisi menggunakan daftar dari array algoritma di atas.
15. **areaData_3002 = new JTextArea();** = Membuat kotak area teks yang luas untuk mencetak tulisan daftar mahasiswa yang masuk.
16. **areaData_3002.setEditable(false);** = Mengunci area teks agar tidak bisa diketik atau dihapus manual oleh pengguna aplikasi.
17. **if (nama_3002.isEmpty() || nim_3002.isEmpty() || prodi_3002.isEmpty()) {** = Pemeriksaan logika apakah ada setidaknya satu kolom form masukan yang terlewat atau dikosongkan.
18. **JOptionPane.showMessageDialog(this, "Semua kolom harus diisi!",**
"Peringatan", JOptionPane.WARNING_MESSAGE); = Memunculkan jendela pesan (pop-up) peringatan agar pengguna mengisi kolom yang kosong.
19. **dataMahasiswa_3002.add(new Mahasiswa_2511533002(nama_3002, nim_3002,**
prodi_3002)); = Membuat objek Mahasiswa baru menggunakan teks isian tadi, dan mendaftarkannya ke dalam wadah penyimpanan ArrayList.
20. **updateTampilanData_3002();** = Memanggil metode terpisah untuk menulis ulang teks yang ada di kotak visualisasi data kiri dengan data terbaru.
21. **if (!dataMahasiswa_3002.isEmpty()) {** = Memeriksa apabila tempat penyimpanan data ArrayList memang ada isinya.

22. **dataMahasiswa_3002.remove(dataMahasiswa_3002.size() - 1);** = Menghapus entitas objek yang menempati indeks paling ujung atau data yang terakhir kali didaftarkan.
23. **updateTampilanData_3002();** = Mencetak ulang kembali tampilan tabel teks di UI.
24. **for (int i_3002 = 0; i_3002 < dataMahasiswa_3002.size(); i_3002++) {** = Perulangan melintasi semua catatan yang direkam ke dalam penyimpanan data.
25. **sb_3002.append((i_3002 + 1)).append(".**
".append(dataMahasiswa_3002.get(i_3002).toString()).append("\n"); = Menambahkan urutan nomor, titik, mahasiswa, lalu dilanjutkan dengan baris baru.
26. **if (dataMahasiswa_3002.isEmpty()) { JOptionPane... return; }** = Memutus aliran program dengan jendela pop-up error apabila pengguna menekan tombol saat belum ada data satupun yang diisi.
27. **ArrayList<Mahasiswa_2511533002> dataCopy_3002 = new**
ArrayList<>(dataMahasiswa_3002); = Menciptakan salinan yang sama persis dari array list milik kita, agar data original/awal tidak terpengaruh secara permanen demi mengakomodasi percobaan tes ulang perbandingan antar algoritma.
28. **String pilihan_3002 = (String) comboAlgo_3002.getSelectedItem();** = Mengamankan status nama opsi jenis sorting apa yang sedang ditunjuk aktif di ComboBox drop-down.
29. **areaProses_3002.setText("==== " + pilihan_3002.toUpperCase() + " ==== \n");** = Membersihkan area layar teks kanan, lalu mengeksekusi pencetakan judul nama algoritma yang dipilih.
30. **if (pilihan_3002.equals("Insertion Sort")) {** = Cabang pengkondisian pertama, mengecek apakah pengguna lebih suka mengoperasikan "Insertion Sort".
31. **for (Mahasiswa_2511533002 m_3002 : dataCopy_3002) {** = Memanfaatkan for loop untuk mengambil satu per satu entitas mahasiswa dari daftar yang sukses terurut sempurna.
32. **SwingUtilities.invokeLater() -> {** = Garis perintah ini memerintahkan kepada CPU supaya sistem inisiasi struktur grafis jendela dilempar menuju ke antrian khusus (Event Dispatching Thread) yang dijamin bebas konflik demi kelancaran komponen UI.
33. **new MahasiswaSortGui_2511533002().setVisible(true);** = Menciptakan objek nyata dari rakitan seluruh tata letak program yang digagas dari baris-baris atas tadi dan menyalakan properti visibilitas grafis ke arah nilai true.

3.4 Output

1. Insertion

Pengurutan Nama Mahasiswa

Input Data Mahasiswa

Nama:

NIM:

Program Studi:

Tambah Data Hapus Data Terakhir

Data Mahasiswa

1. rendi | 251153 | tekomp
2. cia | 251158 | si
3. jeva | 251155 | infor
4. vuas | 251151 | tekomp

Proses Sorting

```

=== INSERTION SORT ===
Awal: [rendi, cia, jeva, vuas]

Langkah 1: [cia, rendi, jeva, vuas]
Langkah 2: [cia, jeva, rendi, vuas]
Langkah 3: [cia, jeva, rendi, vuas]

=== HASIL AKHIR ===
- cia | 251158 | si
- jeva | 251155 | infor
- rendi | 251153 | tekomp
- vuas | 251151 | tekomp

```

Pilih Algoritma

Algoritma: Insertion Sort Mulai Sorting

2. Selection

Pengurutan Nama Mahasiswa

Input Data Mahasiswa

Nama:

NIM:

Program Studi:

Tambah Data Hapus Data Terakhir

Data Mahasiswa

1. rendi | 251153 | tekomp
2. cia | 251158 | si
3. jeva | 251155 | infor
4. vuas | 251151 | tekomp

Proses Sorting

```

=== SELECTION SORT ===
Awal: [rendi, cia, jeva, vuas]

Pass 1: [cia, rendi, jeva, vuas]
Pass 2: [cia, jeva, rendi, vuas]
Pass 3: [cia, jeva, rendi, vuas]

=== HASIL AKHIR ===
- cia | 251158 | si
- jeva | 251155 | infor
- rendi | 251153 | tekomp
- vuas | 251151 | tekomp

```

Pilih Algoritma

Algoritma: Selection Sort Mulai Sorting

3. Bubble

Pengurutan Nama Mahasiswa

Input Data Mahasiswa

Nama:

NIM:

Program Studi:

Tambah Data

Hapus Data Terakhir

Data Mahasiswa

1. rendi | 251153 | tekom
2. cia | 251158 | si
3. jeva | 251155 | infor
4. vuas | 251151 | tekom

Proses Sorting

=== BUBBLE SORT ===
Awal: [rendi, cia, jeva, vuas]

Pass 1: [cia, jeva, rendi, vuas]
Pass 2: [cia, jeva, rendi, vuas]

=== HASIL AKHIR ===
- cia | 251158 | si
- jeva | 251155 | infor
- rendi | 251153 | tekom
- vuas | 251151 | tekom

Pilih Algoritma

Algoritma: Bubble Sort

Mulai Sorting

BAB III

KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa algoritma pengurutan data *sorting* seperti *Insertion Sort*, *Selection Sort*, dan *Bubble Sort* merupakan operasi fundamental yang sangat penting untuk menyusun informasi acak menjadi struktur linier yang teratur dan bermakna. Berbeda dengan data mentah yang proses pencariannya lambat dan tidak efisien, ketiga algoritma ini menawarkan penyelesaian masalah dengan karakteristik manipulasinya masing-masing. *Insertion Sort* bekerja secara intuitif dengan mengambil satu elemen dan menyisipkannya ke posisi yang tepat, *Selection Sort* berfokus pada pencarian nilai ekstrem (terkecil/terbesar) untuk ditukar ke posisi paling awal secara berurutan, sementara *Bubble Sort* mengandalkan pertukaran elemen yang bersebelahan secara terus-menerus hingga nilai terbesar "mengapung" ke ujung akhir. Pemahaman terhadap kerangka dasar dan alur kerja perbandingan pada ketiga algoritma ini sangat krusial, karena memungkinkan penelusuran dan penyusunan data dilakukan secara bertahap dan sistematis.